
YOH PAL LIVE RELEASE 1.0C — Feed, Live Discovery & UX Recovery Patch.

Daniel Macharia <danielmacharia43@gmail.com>

Thu, 9 Jul at 22:47

To: <daniel@ics.ac.ke>

Below is **YOH PAL LIVE RELEASE 1.0C — Feed, Live Discovery & UX Recovery Patch.**

This directly addresses the audit findings that Search and Live Discovery are built but unreachable, feed errors are dead ends, feed pagination is missing, accessibility is absent, and LiveKit drops can hang silently.

YohPal Live — Production Readiness Review.pdf

```
// lib/core/routing/app_routes.dart
class AppRoutes {
  static const discover = '/';
  static const search = '/search';
  static const liveDiscovery = '/live-discovery';
  static const creatorAnalytics = '/creator-profile/analytics';
  static const walletStatus = '/wallet-status';
}

// lib/features/discover/discover_hub_screen.dart
import 'package:flutter/material.dart';
import '../core/routing/app_routes.dart';
import '../video_feed/video_feed_screen.dart';

class DiscoverHubScreen extends StatelessWidget {
  const DiscoverHubScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: const VideoFeedScreen(),
      floatingActionButton: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          FloatingActionButton.small(
            heroTag: 'search',
            tooltip: 'Search',
            onPressed: () => Navigator.pushNamed(context, AppRoutes.search),
            child: const Icon(Icons.search),
          ),
          const SizedBox(height: 12),
          FloatingActionButton.small(
            heroTag: 'live_discovery',
            tooltip: 'Find live streams',
            onPressed: () => Navigator.pushNamed(context, AppRoutes.liveDiscovery),
            child: const Icon(Icons.live_tv),
          ),
        ],
      ),
    );
  }
}

// lib/features/video_feed/video_feed_page.dart
class VideoFeedPage<T> {
  final List<T> items;
  final Object? cursor;
  final bool hasMore;

  const VideoFeedPage({
    required this.items,
    required this.cursor,
    required this.hasMore,
  });
}

// lib/features/video_feed/video_feed_repository.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'video_feed_page.dart';

class VideoFeedRepository {
  final FirebaseFirestore firestore;

  VideoFeedRepository({FirebaseFirestore? firestore})
    : firestore = firestore ?? FirebaseFirestore.instance;
```

```

Future<VideoFeedPage<QueryDocumentSnapshot<Map<String, dynamic>>>> fetchPage({
  DocumentSnapshot? cursor,
  int limit = 25,
}) async {
  Query<Map<String, dynamic>> query = firestore
    .collection('videos')
    .where('visibility', isEqualTo: 'public')
    .orderBy('createdAt', descending: true)
    .limit(limit);

  if (cursor != null) {
    query = query.startAfterDocument(cursor);
  }

  final snap = await query.get();

  return VideoFeedPage(
    items: snap.docs,
    cursor: snap.docs.isEmpty ? cursor : snap.docs.last,
    hasMore: snap.docs.length == limit,
  );
}
}

// lib/features/video_feed/video_feed_error_state.dart
import 'package:flutter/material.dart';

class VideoFeedErrorState extends StatelessWidget {
  final String message;
  final VoidCallback onRetry;

  const VideoFeedErrorState({
    super.key,
    required this.message,
    required this.onRetry,
  });

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Semantics(
        label: 'Video feed error',
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.wifi_off, size: 48),
              const SizedBox(height: 12),
              Text(
                'We could not load your feed.',
                style: Theme.of(context).textTheme.titleMedium,
              ),
              const SizedBox(height: 8),
              Text(message, textAlign: TextAlign.center),
              const SizedBox(height: 16),
              FilledButton.icon(
                onPressed: onRetry,
                icon: const Icon(Icons.refresh),
                label: const Text('Retry'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

// lib/features/video_feed/widgets/feed_video_gesture_layer.dart
import 'package:flutter/material.dart';

class FeedVideoGestureLayer extends StatelessWidget {
  final Widget child;
  final VoidCallback onSingleTap;
  final VoidCallback onDoubleTapLike;
  final VoidCallback onLongPress;
  final String semanticsLabel;

  const FeedVideoGestureLayer({
    super.key,
    required this.child,
    required this.onSingleTap,
    required this.onDoubleTapLike,
    required this.onLongPress,
    required this.semanticsLabel,
  });

```

```

@override
Widget build(BuildContext context) {
  return Semantics(
    label: semanticsLabel,
    button: true,
    child: GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTap: onSingleTap,
      onDoubleTap: onDoubleTapLike,
      onLongPress: onLongPress,
      child: child,
    ),
  );
}
}

// lib/features/video_feed/widgets/mute_button.dart
import 'package:flutter/material.dart';

class MuteButton extends StatelessWidget {
  final bool muted;
  final VoidCallback onToggle;

  const MuteButton({
    super.key,
    required this.muted,
    required this.onToggle,
  });

  @override
  Widget build(BuildContext context) {
    return Semantics(
      label: muted ? 'Unmute video' : 'Mute video',
      button: true,
      child: IconButton(
        tooltip: muted ? 'Unmute' : 'Mute',
        onPressed: onToggle,
        icon: Icon(muted ? Icons.volume_off : Icons.volume_up),
      ),
    );
  }
}

// lib/features/live_streaming/livekit_connection_observer.dart
import 'dart:async';

enum YohPalLiveConnectionState {
  connected,
  reconnecting,
  disconnected,
  failed,
}

class LiveKitConnectionObserver {
  final _controller = StreamController<YohPalLiveConnectionState>.broadcast();

  Stream<YohPalLiveConnectionState> get stream => _controller.stream;

  Timer? _timeout;

  void onConnected() {
    _timeout?.cancel();
    _controller.add(YohPalLiveConnectionState.connected);
  }

  void onConnectionLost() {
    _controller.add(YohPalLiveConnectionState.reconnecting);

    _timeout?.cancel();
    _timeout = Timer(const Duration(seconds: 10), () {
      _controller.add(YohPalLiveConnectionState.failed);
    });
  }

  void onDisconnected() {
    _timeout?.cancel();
    _controller.add(YohPalLiveConnectionState.disconnected);
  }

  void dispose() {
    _timeout?.cancel();
    _controller.close();
  }
}

// lib/features/live_streaming/live_connection_banner.dart

```

```

import 'package:flutter/material.dart';
import 'livekit_connection_observer.dart';

class LiveConnectionBanner extends StatelessWidget {
  final YohPalLiveConnectionState state;
  final VoidCallback onRetry;

  const LiveConnectionBanner({
    super.key,
    required this.state,
    required this.onRetry,
  });

  @override
  Widget build(BuildContext context) {
    if (state == YohPalLiveConnectionState.connected) {
      return const SizedBox.shrink();
    }

    final failed = state == YohPalLiveConnectionState.failed;

    return Material(
      color: failed ? Colors.red.shade700 : Colors.orange.shade700,
      child: SafeArea(
        bottom: false,
        child: ListTile(
          title: Text(
            failed ? 'Live connection lost' : 'Reconnecting to live stream...',
            style: const TextStyle(color: Colors.white),
          ),
          trailing: failed
            ? TextButton(
                onPressed: onRetry,
                child: const Text(
                  'Retry',
                  style: TextStyle(color: Colors.white),
                ),
              )
            : const SizedBox(
                height: 18,
                width: 18,
                child: CircularProgressIndicator(strokeWidth: 2),
              ),
        ),
      ),
    );
  }
}

// test/navigation/live_search_routes_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:yohpal_live/core/routing/app_routes.dart';

void main() {
  test('Search and Live Discovery routes are declared', () {
    expect(AppRoutes.search, '/search');
    expect(AppRoutes.liveDiscovery, '/live-discovery');
  });
}

flutter analyze
flutter test

```

Developer Evidence Required

1. Git branch
2. Commit hash
3. Search navigation screenshot
4. Live Discovery navigation screenshot
5. Feed retry screenshot
6. Pagination proof
7. Mute/unmute proof
8. Double-tap-like proof
9. Accessibility/Semantics proof
10. LiveKit reconnect proof
11. flutter analyze output
12. flutter test output

Next Smart Move

Execute **YOHPAL LIVE RELEASE 1.0D — Codebase Cleanup, Tests & Release Gate Patch** to remove duplicate video pipelines, delete dead services, consolidate analytics, add route-reachability CI, and expand automated tests for feed, auth, wallet, checkout, and live streaming.

